

LAPORAN PRAKTIKUM BIG DATA
MINGGU 1: PENGENALAN BIG DATA & LINGKUNGAN KERJA, SERTA
PENGOLAHAN DATASET BIG DATA MENGGUNAKAN PYTHON



OLEH:

ALFARIS AULIA RAHMAN

2411533006

DOSEN PENGAMPU:

AFDHAL DINILHAK, S.KOMP., M.KOM.

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

A. Pendahuluan

Big Data tidak hanya ditentukan oleh ukuran file, tetapi oleh hubungan antara data, cara penyimpanan, metode pemrosesan, dan sumber daya mesin. Data dapat dikategorikan sebagai Big Data ketika metode pengolahan biasa mulai mengalami kendala, seperti penggunaan memori yang besar, waktu pemrosesan yang lama, format data yang beragam, kualitas data yang rendah, atau kebutuhan analisis yang cepat.

Pada praktikum ini digunakan dataset NYC TLC Yellow Taxi Trip Records. Dataset utama yang digunakan adalah `yellow_tripdata_2023-07.parquet`, sedangkan dataset Januari 2023 digunakan sebagai pembanding. Data tersebut memiliki jutaan baris perjalanan taksi dan digunakan untuk menguji pemilihan kolom, optimasi tipe data, chunking, serta pemrosesan menggunakan Spark local mode.

Laporan ini membahas proses pengolahan dataset Juli dari tahap inspeksi metadata sampai visualisasi. Selain itu, dilakukan perbandingan pola permintaan per jam antara Januari dan Juli, analisis kinerja pemrosesan, refleksi 5V, serta jawaban pertanyaan evaluasi pada modul.

B. Tujuan

1. Menyiapkan notebook Colab dan mendokumentasikan spesifikasi lingkungan.
2. Memeriksa integritas dataset, meliputi ukuran file, jumlah baris, dan skema kolom.
3. Memuat, membersihkan, dan mengagregasi jutaan baris dengan strategi hemat memori.
4. Memproses file yang lebih besar dari RAM menggunakan chunking.
5. Menghasilkan tabel agregasi per jam dan per metode pembayaran dalam format CSV dan Parquet.

6. Menjalankan agregasi dengan PySpark local mode serta menjelaskan perbedaannya dengan pandas.

C. Dasar Teori dan Dataset

1. Karakteristik 5V Big Data

a. Volume

Volume berkaitan dengan jumlah data yang harus disimpan dan diproses. Dataset Juli memiliki 2.907.108 baris dan ketika semua kolom dibaca membutuhkan memori DataFrame sebesar 502,5 MB.

b. Velocity

Velocity berkaitan dengan kecepatan data masuk dan kebutuhan hasil secara real-time. Dataset pada praktikum ini berupa file bulanan, bukan data streaming, sehingga dimensi velocity belum menjadi masalah utama.

c. Variety

Variety berkaitan dengan keberagaman format atau struktur data. Pada praktikum ini dibandingkan format Parquet dan CSV. Parquet lebih sesuai untuk analisis karena bersifat columnar.

d. Veracity

Veracity berkaitan dengan kualitas dan kepercayaan terhadap data. Dataset Juli memiliki nilai jarak nol, durasi negatif, durasi terlalu lama, serta total tarif negatif atau nol.

e. Value

Value berkaitan dengan manfaat hasil pengolahan bagi pengambilan keputusan. Agregasi jumlah perjalanan per jam dapat digunakan untuk membantu penjadwalan armada taksi.

2. Parquet dan CSV

Parquet menyimpan data berdasarkan kolom dan memiliki skema data. Karena itu, pembaca Parquet dapat mengambil hanya kolom yang diperlukan. Parquet juga menggunakan encoding dan kompresi columnar. CSV menyimpan data dalam bentuk teks per baris. Setiap nilai harus diparse kembali ketika dibaca dan CSV tidak mendukung column pruning seefisien Parquet.

3. Chunking

Chunking adalah teknik membaca file secara bertahap. Data tidak dimuat sekaligus, tetapi dibaca dalam beberapa potongan menggunakan parameter chunksize. Keuntungan chunking adalah penggunaan memori lebih kecil. Kekurangannya adalah kode lebih kompleks dan tidak semua agregasi dapat digabungkan secara langsung. Agregasi seperti count, sum, min, dan max mudah digabungkan, sedangkan median eksak lebih sulit.

4. Spark Local Mode

Spark local mode menjalankan Spark pada satu mesin menggunakan local[*]. Transformasi seperti filter, withColumn, dan groupBy bersifat lazy. Eksekusi baru dilakukan ketika action seperti count(), show(), atau write() dipanggil.

5. Dataset yang Digunakan

D. Langkah Pengerjaan Praktikum

1. Verifikasi Lingkungan Kerja

Pada tahap ini dilakukan pemeriksaan terhadap spesifikasi runtime yang digunakan, meliputi versi Python, versi library, jumlah core CPU, kapasitas RAM, kapasitas disk, dan versi Java. Informasi ini diperlukan agar hasil pengukuran waktu dan memori dapat dipahami sesuai kondisi mesin yang digunakan.

```
import sys, platform, multiprocessing

print("Python :", sys.version.split()[0])
print("OS :", platform.platform())
print("CPU core:", multiprocessing.cpu_count())

for nama in ["pandas", "numpy", "pyarrow", "matplotlib"]:
    try:
        mod = __import__(nama)
        print(f"{nama:11s}: {mod.__version__}")
    except ImportError:
        print(f"{nama:11s}: BELUM TERPASANG")

Python : 3.13.15
OS : Linux-6.6.122+-x86_64-with-glibc2.39
CPU core: 2
pandas : 2.2.3
numpy : 2.1.3
pyarrow : 23.0.1
matplotlib : 3.10.0

ifree -h # kapasitas dan sisa RAM
lfd -h /content # kapasitas dan sisa disk runtime
ljava -version # dibutuhkan PySpark pada langkah K-8

          total      used      free   shared  buff/cache   available
Mem:      12Gi      3.3Gi      5.6Gi      2.1Mi      4.1Gi      9.4Gi
Swap:      0B           0B           0B
Filesystem      Size  Used Avail Use% Mounted on
overlay         108G   22G   87G   21% /
[0.002s] [warning] [os,container] Cgroup memory controller path at '/sys/fs/cgroup' seems to have moved to '/../../jupyter-children', detected limits won't be accurate
[0.002s] [warning] [os,container] Cgroup cpu controller path at '/sys/fs/cgroup' seems to have moved to '/../../jupyter-children', detected limits won't be accurate
openjdk version "21.0.12" 2026-07-21
OpenJDK Runtime Environment (build 21.0.12+8-1-24.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12+8-1-24.04-Ubuntu, mixed mode, sharing)
```

2. Menghubungkan Google Drive dan Menyiapkan Struktur Folder

Google Drive dihubungkan ke runtime Google Colab sebagai tempat penyimpanan dataset dan artefak hasil praktikum. Dataset dibaca dari folder MyDrive/BigData/Praktikum1.

```
from google.colab import drive
drive.mount("/content/drive")

import os
DIR_KERJA = "/content/data"
DIR_SIMPAN = "/content/drive/MyDrive/BigData/Praktikum1"
os.makedirs(DIR_KERJA, exist_ok=True)
os.makedirs(DIR_SIMPAN, exist_ok=True)
print(os.listdir(DIR_SIMPAN))

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
['yellow_tripdata_2023-01.parquet', 'agregasi_per_jam.csv', 'agregasi_per_jam.parquet', 'distribusi_per_jam.png', 'pengukuran_kinerja.csv', 'bersih_snappy.parquet', 'bersih_snappy.parquet']
```

3. Menyiapkan Alat Ukur Waktu dan Memori

Pada tahap ini dibuat fungsi pengukuran untuk mencatat durasi proses dan perubahan penggunaan memori. Durasi diukur menggunakan `time.perf_counter()`, sedangkan penggunaan memori proses diukur menggunakan RSS dari `psutil`.

```
[4]
✓ Os
import time, os, psutil

proses = psutil.Process(os.getpid())
catatan = []

def rss_mb():
    return proses.memory_info().rss / 1024**2

def ukur(label, fungsi):
    """Menjalankan fungsi tanpa argumen, mencatat durasi dan pertumbuhan memori."""
    m0, t0 = rss_mb(), time.perf_counter()
    hasil = fungsi()
    detik = time.perf_counter() - t0
    delta = rss_mb() - m0
    catatan.append({"langkah": label, "detik": round(detik, 2),
                    "delta_rss_mb": round(delta, 1)})
    print(f"[{label}] {detik:.2f} s | RSS {delta:+.1f} MB")
    return hasil
```

4. Menyiapkan Dataset dan Melakukan Inspeksi Awal

Dataset `yellow_tripdata_2023-07.parquet` dibaca langsung dari Google Drive. Metadata Parquet diperiksa untuk mengetahui ukuran file, jumlah baris, jumlah kolom, jumlah row group, dan nama kolom tanpa memuat seluruh data ke memori.

```
[5]
✓ Os
import urllib.request

URL = ("https://d37ci6vzurychx.cloudfront.net/trip-data/"
      "yellow_tripdata_2023-07.parquet")
PATH = os.path.join(DIR_KERJA, "yellow_tripdata_2023-07.parquet")

if not os.path.exists(PATH):
    ukur("unduh dataset", lambda: urllib.request.urlretrieve(URL, PATH))

print("Ukuran file:", round(os.path.getsize(PATH) / 1024**2, 1), "MB")

[unduh dataset] 0.48 s | RSS +1.8 MB
Ukuran file: 45.5 MB

[6]
✓ Os
import pyarrow.parquet as pq

meta = pq.ParquetFile(PATH).metadata
print("Jumlah baris :", f"{meta.num_rows:,}")
print("Jumlah kolom :", meta.num_columns)
print("Row group   :", meta.num_row_groups)
print("Nama kolom   :", pq.ParquetFile(PATH).schema.names)

Jumlah baris : 3,066,766
Jumlah kolom : 19
Row group    : 1
Nama kolom   : ['VendorID', 'tpep_pickup_datetime', 'tpep_dropoff_datetime', 'passenger_count', 'trip_distance', 'RatecodeID', 'store_and_fwd_flag', 'PULocationID', 'DOLC']
```

5. Memuat Data dan Mengukur Biaya Memori

Data dibaca dalam dua pendekatan, yaitu dengan membaca seluruh kolom dan dengan membaca hanya kolom yang diperlukan. Kedua pendekatan tersebut dibandingkan berdasarkan waktu pembacaan, ukuran memori DataFrame, dan pertumbuhan RSS. Selanjutnya dilakukan optimasi

tipe data menggunakan downcast dan category untuk mengurangi penggunaan memori.

```
[7]
✓ 6s
import pandas as pd

KOLOM = ["tpep_pickup_datetime", "tpep_dropoff_datetime",
         "passenger_count", "trip_distance", "payment_type",
         "fare_amount", "tip_amount", "total_amount"]

penuh = ukur("baca semua kolom", lambda: pd.read_parquet(PATH))
print("Dimensi:", penuh.shape)
print("Memori :", round(penuh.memory_usage(deep=True).sum() / 1024**2, 1), "MB")

del penuh
import gc; gc.collect()

df = ukur("baca kolom terpilih", lambda: pd.read_parquet(PATH, columns=KOLOM))
print("Dimensi:", df.shape)
print("Memori :", round(df.memory_usage(deep=True).sum() / 1024**2, 1), "MB")
df.head()
```

[baca semua kolom] 2.62 s | RSS +495.4 MB
Dimensi: (3066766, 19)
Memori : 565.6 MB
[baca kolom terpilih] 2.32 s | RSS +228.7 MB
Dimensi: (3066766, 8)
Memori : 187.2 MB

	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	payment_type	fare_amount	tip_amount	total_amount
0	2023-01-01 00:32:10	2023-01-01 00:40:36	1.0	0.97	2	9.3	0.00	14.30
1	2023-01-01 00:55:08	2023-01-01 01:01:27	1.0	1.10	1	7.9	4.00	16.90
2	2023-01-01 00:25:04	2023-01-01 00:37:49	1.0	2.51	1	14.9	15.00	34.90
3	2023-01-01 00:03:48	2023-01-01 00:13:25	0.0	1.90	1	12.1	0.00	20.85
4	2023-01-01 00:10:29	2023-01-01 00:21:19	1.0	1.43	1	11.4	3.28	19.68

```
df.info(memory_usage="deep")
df.memory_usage(deep=True).sort_values(ascending=False) / 1024**2 # MB per kolom
```

... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 3066766 entries, 0 to 3066765
Data columns (total 8 columns):
Column Dtype

0 tpep_pickup_datetime datetime64[us]
1 tpep_dropoff_datetime datetime64[us]
2 passenger_count float64
3 trip_distance float64
4 payment_type int64
5 fare_amount float64
6 tip_amount float64
7 total_amount float64
dtypes: datetime64[us](2), float64(5), int64(1)
memory usage: 187.2 MB

	0
tpep_pickup_datetime	23.397568
payment_type	23.397568
tpep_dropoff_datetime	23.397568
passenger_count	23.397568
trip_distance	23.397568
tip_amount	23.397568
fare_amount	23.397568
total_amount	23.397568
Index	0.000126

dtype: float64

```
sebelum = df.memory_usage(deep=True).sum() / 1024**2

df["passenger_count"] = pd.to_numeric(df["passenger_count"], downcast="float")
df["payment_type"] = df["payment_type"].astype("int8").astype("category")
for kol in ["trip_distance", "fare_amount", "tip_amount", "total_amount"]:
    df[kol] = pd.to_numeric(df[kol], downcast="float")

sesudah = df.memory_usage(deep=True).sum() / 1024**2
print(f"{sebelum:.1f} MB -> {sesudah:.1f} MB (hemat {100*(1-sesudah/sebelum):.1f}%)")
df.dtypes

187.2 MB -> 119.9 MB (hemat 35.9%)

0
tpep_pickup_datetime    datetime64[us]
tpep_dropoff_datetime   datetime64[us]
passenger_count          float32
trip_distance            float64
payment_type             category
fare_amount              float32
tip_amount               float32
total_amount             float32

dtype: object
```

6. Memeriksa Kualitas dan Membersihkan Data

Kualitas data diperiksa melalui nilai kosong dan beberapa kondisi anomali. Baris yang memiliki jarak tidak valid, durasi tidak valid, durasi terlalu lama, atau total_amount tidak positif dikeluarkan dari analisis.

Threshold yang digunakan adalah:

- $0 < \text{trip_distance} < 100$
- $1 \leq \text{durasi_menit} \leq 180$
- $\text{total_amount} > 0$

```
[10]
df["durasi_menit"] = (df["tpep_dropoff_datetime"] -
df["tpep_pickup_datetime"]).dt.total_seconds() / 60

print(df[["trip_distance", "durasi_menit", "total_amount"]].describe())
print("\nNilai kosong per kolom:\n", df.isna().sum())

anomali = {
    "jarak <= 0" : (df["trip_distance"] <= 0).sum(),
    "durasi <= 0" : (df["durasi_menit"] <= 0).sum(),
    "durasi > 180 menit": (df["durasi_menit"] > 180).sum(),
    "total_amount <= 0" : (df["total_amount"] <= 0).sum(),
}
for k, v in anomali.items():
    print(f"{k:22s}: {v:,} baris ({100*v/len(df):.2f}%)")
```

	trip_distance	durasi_menit	total_amount
count	3.066766e+06	3.066766e+06	3.066766e+06
mean	3.847342e+00	1.566900e+01	2.702038e+01
std	2.495838e+02	4.259435e+01	2.212666e+01
min	0.000000e+00	-2.920000e+01	-7.510000e+02
25%	1.060000e+00	7.116667e+00	1.540000e+01
50%	1.800000e+00	1.151667e+01	2.016000e+01
75%	3.330000e+00	1.830000e+01	2.870000e+01
max	2.589281e+05	1.002918e+04	1.169400e+03

```

Nilai kosong per kolom:
tpep_pickup_datetime    0
tpep_dropoff_datetime   0
passenger_count         71743
trip_distance           0
payment_type            0
fare_amount             0
tip_amount              0
total_amount            0
durasi_menit            0
dtype: int64
jarak <= 0 : 45,862 baris (1.50%)
durasi <= 0 : 1,121 baris (0.04%)
durasi > 180 menit : 3,048 baris (0.10%)
total_amount <= 0 : 25,772 baris (0.84%)

```

```
Add text cell
layak = (
    (df["trip_distance"] > 0) & (df["trip_distance"] < 100) &
    df["durasi_menit"].between(1, 180) &
    (df["total_amount"] > 0)
)
bersih = df.loc[layak].copy()
print(f"{len(df):,} baris -> {len(bersih):,} baris (dibuang {len(df)-len(bersih):,})")
```

```
3,066,766 baris -> 2,986,911 baris (dibuang 79,855)
```

7. Memproses Data dengan Teknik Chunking

Data bersih ditulis ke dalam format CSV untuk mensimulasikan file yang lebih besar. CSV kemudian dibaca secara bertahap menggunakan chunksize sehingga hanya satu potongan data yang berada di memori pada satu waktu. Tahap ini digunakan untuk membandingkan ukuran Parquet dan CSV serta mengukur waktu dan penggunaan memori selama agregasi chunking.

```
[12] ✓ 48s CSV = os.path.join(DIR_KERJA, "trips.csv")
ukur("tulis CSV", lambda: bersih.to_csv(CSV, index=False))

print("Parquet:", round(os.path.getsize(PATH) / 1024**2, 1), "MB")
print("CSV      :", round(os.path.getsize(CSV) / 1024**2, 1), "MB")

[tulis CSV] 48.09 s | RSS +5.9 MB
Parquet: 45.5 MB
CSV      : 226.2 MB

[13] ✓ 4s def agregasi_bertahap(path, ukuran_chunk=500_000):
    jumlah_baris = 0
    total_nilai = 0.0
    per_jam = {}
    potongan = pd.read_csv(
        path,
        usecols=["tpep_pickup_datetime", "total_amount"],
        parse_dates=["tpep_pickup_datetime"],
        chunksize=ukuran_chunk)
    for chunk in potongan:
        jumlah_baris += len(chunk)
        total_nilai += chunk["total_amount"].sum()
        for jam, sub in chunk.groupby(chunk["tpep_pickup_datetime"].dt.hour):
            n, nilai = per_jam.get(jam, (0, 0.0))
            per_jam[jam] = (n + len(sub), nilai + sub["total_amount"].sum())
    return jumlah_baris, total_nilai, per_jam

baris, total, per_jam_chunk = ukur(
    "agregasi chunking", lambda: agregasi_bertahap(CSV))
print(f'{baris:,} baris | rata-rata total_amount = {total/baris:.2f}')

[agregasi chunking] 4.26 s | RSS +19.4 MB
2,986,911 baris | rata-rata total_amount = 27.33
```

8. Menjalankan PySpark dalam Local Mode

Dataset Juli diproses menggunakan PySpark dengan konfigurasi local[*]. Spark digunakan untuk menghitung jumlah baris, melakukan filter data, mengagregasi perjalanan per jam, mengelompokkan data berdasarkan payment_type, dan menampilkan rencana eksekusi menggunakan explain(). Pada tahap ini juga diamati konsep lazy evaluation, partition, shuffle, dan Exchange.

```
[12] ✓ 8s !pip install -q pyspark==3.5.1
!java -version

[0.001s][warning][os,container] Cgroup memory controller path at '/sys/fs/cgroup' seems to have moved to '/../../jupyter-children', detected limits won't be accurate
[0.001s][warning][os,container] Cgroup cpu controller path at '/sys/fs/cgroup' seems to have moved to '/../../jupyter-children', detected limits won't be accurate
openjdk version "21.0.12" 2026-07-21
OpenJDK Runtime Environment (build 21.0.12+8-1-24.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12+8-1-24.04-Ubuntu, mixed mode, sharing)
```

```
[15] ✓ 15s from pyspark.sql import SparkSession, functions as F

spark = (SparkSession.builder
    .appName("BD-P01")
    .master("local[*]") # semua core mesin ini
    .config("spark.driver.memory", "4g")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate())

spark.sparkContext.setLogLevel("WARN")
print("Spark:", spark.version)

Spark: 3.5.1
```

[16]
✓ 25s

```
sdf = spark.read.parquet(PATH)
sdf.printSchema()
print("Jumlah baris:", ukur("spark count", lambda: sdf.count()))

sdf_bersih = (sdf
    .withColumn(
        "durasi_menit",
        (F.unix_timestamp("tpep_dropoff_datetime")
         - F.unix_timestamp("tpep_pickup_datetime")) / 60)
    .filter((F.col("trip_distance") > 0) & (F.col("trip_distance") < 100))
    .filter((F.col("durasi_menit") >= 1) & (F.col("durasi_menit") <= 180))
    .filter(F.col("total_amount") > 0))

hasil_spark = (sdf_bersih
    .withColumn("jam", F.hour("tpep_pickup_datetime"))
    .groupBy("jam")
    .agg(F.count("*").alias("jumlah"),
        F.round(F.avg("trip_distance"), 2).alias("rata_jarak"),
        F.round(F.avg("total_amount"), 2).alias("rata_tarif"))
    .orderBy("jam"))

ukur("Spark agregasi", lambda: hasil_spark.show(24, truncate=False))
```

▼

```
root
|-- VendorID: long (nullable = true)
|-- tpep_pickup_datetime: timestamp_ntz (nullable = true)
|-- tpep_dropoff_datetime: timestamp_ntz (nullable = true)
|-- passenger_count: double (nullable = true)
|-- trip_distance: double (nullable = true)
|-- RatecodeID: double (nullable = true)
|-- store_and_fwd_flag: string (nullable = true)
|-- PULocationID: long (nullable = true)
|-- DOLocationID: long (nullable = true)
|-- payment_type: long (nullable = true)
|-- fare_amount: double (nullable = true)
|-- extra: double (nullable = true)
|-- mta_tax: double (nullable = true)
|-- tip_amount: double (nullable = true)
|-- tolls_amount: double (nullable = true)
|-- improvement_surcharge: double (nullable = true)
|-- total_amount: double (nullable = true)
|-- congestion_surcharge: double (nullable = true)
|-- airport_fee: double (nullable = true)
```

```
[spark count] 9.23 s | RSS +0.0 MB
Jumlah baris: 3066766
```

```

[spark count] 9.23 s | RSS +0.0 MB
Jumlah baris: 3066766
+-----+-----+-----+-----+
|jam|jumlah|rata_jarak|rata_tarif|
+-----+-----+-----+-----+
|0|82289|4.04|28.86|
|1|57766|3.53|26.45|
|2|40445|3.28|25.06|
|3|26146|3.58|26.26|
|4|16618|4.79|31.72|
|5|16940|6.03|36.75|
|6|42300|4.8|30.63|
|7|84569|3.69|26.89|
|8|114064|3.08|25.21|
|9|127988|3.09|25.51|
|10|140230|3.13|25.53|
|11|150539|3.0|25.16|
|12|165714|3.11|25.61|
|13|174183|3.29|26.43|
|14|186741|3.57|28.04|
|15|191392|3.41|27.42|
|16|190689|3.51|29.58|
|17|204399|3.3|28.62|
|18|210858|2.94|26.69|
|19|188347|3.25|27.43|
|20|161789|3.44|26.98|
|21|157770|3.61|27.59|
|22|143800|3.85|28.61|
|23|111335|4.23|29.98|
+-----+-----+-----+-----+

[Spark agregasi] 6.09 s | RSS +0.0 MB

```

```

(17) ✓ 5s
sdf_bersih.createOrReplaceTempView("trips")
spark.sql("""
  SELECT payment_type,
         COUNT(*) AS jumlah,
         ROUND(AVG(total_amount), 2) AS rata_tarif,
         ROUND(AVG(tip_amount), 2) AS rata_tip
  FROM trips
  GROUP BY payment_type
  ORDER BY jumlah DESC
""").show()

hasil_spark.explain() # rencana eksekusi - perhatikan Exchange/HashAggregate
spark.stop()          # bebaskan sumber daya sebelum lanjut

+-----+-----+-----+
|payment_type|jumlah|rata_tarif|rata_tip|
+-----+-----+-----+
|1|2385064|28.1|4.16|
|2|512987|23.58|0.0|
|0|64186|29.82|4.06|
|4|15570|24.11|0.02|
|3|9184|22.63|0.0|
+-----+-----+-----+

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [jam#84 ASC NULLS FIRST], true, 0
   +- Exchange rangepartitioning(jam#84 ASC NULLS FIRST, 8), ENSURE_REQUIREMENTS, [plan_id=206]
      +- HashAggregate(keys=[jam#84], functions=[count(1), avg(trip_distance#4), avg(total_amount#16)])
         +- Exchange hashpartitioning(jam#84, 8), ENSURE_REQUIREMENTS, [plan_id=203]
            +- HashAggregate(keys=[jam#84], functions=[partial_count(1), partial_avg(trip_distance#4), partial_avg(total_amount#16)])
               +- Project [trip_distance#4, total_amount#16, hour(tpep_pickup_datetime#1, Some(Etc/UTC)) AS jam#84]
                  +- Filter ((((((isnotnull(trip_distance#4) AND isnotnull(total_amount#16)) AND (trip_distance#4 > 0.0)) AND (trip_distance#4 < 100.0)) AND ((cast((unix_1
                     +- FileScan parquet [tpep_pickup_datetime#1,tpep_dropoff_datetime#2,trip_distance#4,total_amount#16] Batched: true, DataFilters: [isnotnull(trip_dista

```

9. Melakukan Agregasi Akhir dan Menyimpan Artefak

Data bersih diagregasi berdasarkan jam pickup. Hasil agregasi berisi jumlah perjalanan, rata-rata jarak, rata-rata durasi, rata-rata tarif, dan median tarif.

Hasil pengolahan disimpan dalam beberapa artefak:

- agregasi_per_jam.csv
- agregasi_per_jam.parquet

- bersih_juli.parquet
- pengukuran_kinerja.csv

```
[18]
✓ 1s

bersih["jam"] = bersih["tpep_pickup_datetime"].dt.hour

agregasi = (bersih
            .groupby("jam")
            .agg(jumlah_perjalanan=("total_amount", "size"),
                rata_jarak=("trip_distance", "mean"),
                rata_durasi=("durasi_menit", "mean"),
                rata_tarif=("total_amount", "mean"),
                median_tarif=("total_amount", "median"))
            .round(2)
            .reset_index())

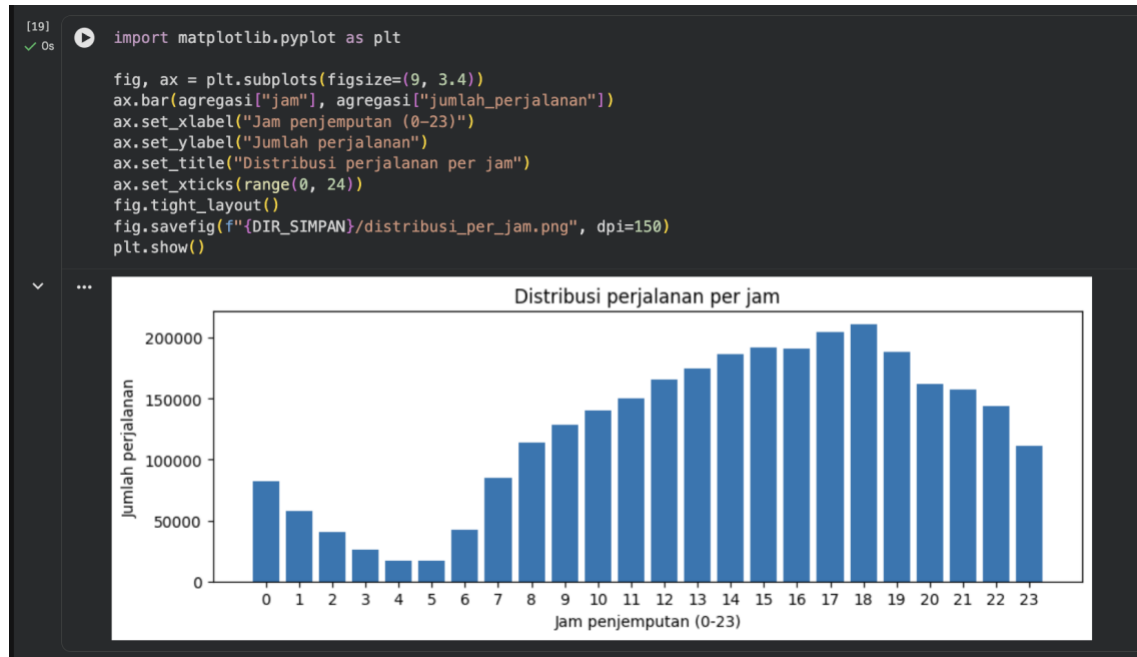
agregasi.to_csv(f"{DIR_SIMPAN}/agregasi_per_jam.csv", index=False)
agregasi.to_parquet(f"{DIR_SIMPAN}/agregasi_per_jam.parquet", index=False)
pd.DataFrame(catatan).to_csv(f"{DIR_SIMPAN}/pengukuran_kinerja.csv", index=False)

agregasi
```

	jam	jumlah_perjalanan	rata_jarak	rata_durasi	rata_tarif	median_tarif
0	0	82289	4.04	13.57	28.860001	21.600000
1	1	57766	3.53	12.63	26.450001	21.200001
2	2	40445	3.28	12.07	25.059999	20.520000
3	3	26146	3.58	12.15	26.260000	21.000000
4	4	16618	4.79	13.52	31.719999	22.700001
5	5	16940	6.03	14.70	36.750000	22.040001
6	6	42300	4.80	14.49	30.629999	18.480000
7	7	84569	3.69	14.42	26.889999	18.480000
8	8	114064	3.08	14.33	25.209999	19.100000
9	9	127988	3.09	14.23	25.510000	19.250000
10	10	140230	3.13	14.25	25.530001	18.900000
11	11	150539	3.00	14.28	25.160000	18.900000
12	12	165714	3.11	14.40	25.610001	18.900000
13	13	174183	3.29	15.06	26.430000	19.219999
14	14	186741	3.57	16.39	28.040001	19.570000
15	15	191392	3.41	16.42	27.420000	19.320000
16	16	190689	3.51	16.52	29.580000	21.400000
17	17	204399	3.30	15.98	28.620001	21.480000
18	18	210858	2.94	14.24	26.690001	21.000000
19	19	188347	3.25	13.64	27.430000	21.000000
20	20	161789	3.44	13.24	26.980000	20.410000
21	21	157770	3.61	13.30	27.590000	20.520000
22	22	143800	3.85	13.75	28.610001	21.360001
23	23	111335	4.23	13.97	29.980000	22.000000

10.Membuat Visualisasi Hasil

Agregasi per jam divisualisasikan menggunakan diagram batang. Grafik ini digunakan untuk melihat perubahan jumlah perjalanan sepanjang hari dan menentukan jam dengan permintaan tertinggi serta terendah.



E. Analisis Hasil

1. Biaya membaca data. Berapa rasio waktu dan memori antara membaca semua kolom dan membaca kolom terpilih? Apakah rasio itu sebanding dengan rasio jumlah kolom? Jika tidak, mengapa?
2. Efek optimasi tipe. Berapa persen memori yang Anda hemat? Kolom mana yang paling besar kontribusinya? Apa risiko yang Anda terima sebagai gantinya?
3. Format penyimpanan. Berapa kali lebih besar CSV dibanding Parquet pada data yang sama? Jelaskan dua penyebabnya.
4. Chunking. Bandingkan `delta_rss_mb` antara pemuatan penuh dan agregasi chunking. Apa yang Anda korbakan untuk memperoleh hemat memori itu?

5. pandas vs Spark. Mana yang lebih cepat pada data ini? Pada ukuran berapa perkiraan Anda urutannya akan berbalik, dan atas dasar apa Anda memperkirakan demikian?
6. Kualitas data. Berapa persen baris yang dibuang? Apakah penghapusan itu berpotensi menghasilkan bias sistematis pada kesimpulan (mis. trip_distance ikut terbuang)?

Jawab:

1. Berdasarkan hasil yang diperoleh, pembacaan semua kolom membutuhkan waktu 4,50 detik, sedangkan pembacaan delapan kolom terpilih hanya membutuhkan 0,69 detik. Artinya, pembacaan semua kolom membutuhkan waktu sekitar 6,52 kali lebih lama. Dari sisi memori DataFrame, semua kolom menggunakan 502,5 MB, sedangkan kolom terpilih menggunakan 177,4 MB atau sekitar 2,83 kali lebih kecil. Rasio tersebut tidak sama dengan rasio jumlah kolom, yaitu 19 dibanding 8 atau 2,38 kali, karena waktu dan memori juga dipengaruhi proses parsing, dekompresi, buffer pembaca, dan alokasi memori.
2. Berdasarkan hasil optimasi, penggunaan memori turun dari 177,4 MB menjadi 113,7 MB. Dengan demikian, penghematan memori yang diperoleh adalah 35,9%. Penghematan terbesar berasal dari perubahan beberapa kolom numerik dari float64 menjadi float32, seperti passenger_count, fare_amount, tip_amount, dan total_amount, serta perubahan payment_type menjadi category. Risiko yang saya terima adalah berkurangnya presisi angka pada float32, sehingga tipe tersebut sebaiknya digunakan untuk agregasi dan bukan untuk perhitungan tagihan atau audit keuangan.
3. Berdasarkan hasil pengukuran, file CSV bersih berukuran 213,6 MB, sedangkan file Parquet berukuran 46,1 MB. Dengan demikian, CSV berukuran sekitar 4,63 kali lebih besar daripada Parquet. Hal ini terjadi

karena CSV menyimpan setiap nilai sebagai teks dan tidak menggunakan encoding kolumnar. Selain itu, Parquet menggunakan schema typed, column pruning, dan kompresi sehingga lebih sesuai untuk beban kerja analitik.

4. Pada saat membaca semua kolom, pertumbuhan RSS yang tercatat adalah 495,5 MB. Sementara itu, agregasi menggunakan chunking hanya menghasilkan delta RSS sebesar 29,3 MB. Hal ini menunjukkan bahwa chunking sangat membantu mengurangi penggunaan memori karena hanya satu potongan data yang berada di memori pada satu waktu. Namun, saya harus mengorbankan kesederhanaan kode dan waktu tambahan untuk menulis data menjadi CSV. Agregasi chunking Juli membutuhkan waktu 5,03 detik untuk memproses 2.817.657 baris.
5. Berdasarkan hasil eksekusi, agregasi menggunakan chunking pandas membutuhkan waktu 5,03 detik, sedangkan agregasi Spark membutuhkan waktu 5,96 detik. Dengan demikian, pandas lebih cepat sekitar 0,93 detik pada pengujian ini. Namun, perbandingan tersebut belum sepenuhnya setara karena pandas memproses CSV menggunakan chunking, sedangkan Spark membaca Parquet secara langsung. Menurut saya, Spark baru akan lebih menguntungkan ketika ukuran data sudah lebih besar daripada kapasitas RAM, harus diproses pada beberapa mesin, atau membutuhkan pemrosesan paralel dalam jumlah besar.
6. Dataset Juli memiliki 2.907.108 baris sebelum pembersihan dan 2.817.657 baris setelah pembersihan. Dengan demikian, sebanyak 89.451 baris atau 3,08% data dibuang. Baris tersebut memiliki jarak tidak valid, durasi tidak valid, durasi lebih dari 180 menit, atau total_amount yang tidak positif. Menurut saya, penghapusan data tersebut berpotensi menghasilkan bias apabila baris anomali tidak tersebar secara acak. Misalnya, jika perjalanan dengan jarak ekstrem lebih banyak berasal dari rute bandara, maka

penghapusannya dapat membuat rata-rata jarak dan tarif terlihat lebih rendah.

F. Studi Kasus

1. Deskripsi Studi Kasus

Studi kasus ini membahas penjadwalan armada taksi berdasarkan pola permintaan perjalanan. Operator taksi memiliki 1.200 armada dan ingin mengetahui waktu dengan permintaan tertinggi, perbedaan pola antara hari kerja dan akhir pekan, serta apakah perjalanan pada jam sibuk lebih pendek dan lebih menguntungkan per menit. Data yang digunakan adalah data bersih dari J-6. Pada tahap analisis, saya menambahkan kolom jam pickup, hari, status akhir pekan, dan tarif kotor per menit menggunakan rumus:

$$\text{tarif_per_menit} = \text{total_amount} / \text{durasi_menit}$$

2. Pertanyaan 1: Jam dengan Permintaan Tertinggi dan Terendah

Berdasarkan hasil agregasi, permintaan perjalanan tertinggi terjadi pada jam 18.00 dengan jumlah 210.858 perjalanan. Permintaan terendah terjadi pada jam 04.00 dengan jumlah 16.618 perjalanan. Hasil ini menunjukkan bahwa operator perlu memberikan perhatian lebih besar pada periode sore hingga awal malam, sedangkan kebutuhan armada pada jam 04.00 cenderung lebih rendah.

▼	Permintaan tertinggi: jam 18.00 (210,858 perjalanan).			
	... Permintaan terendah: jam 04.00 (16,618 perjalanan).			
	jam	jumlah_perjalanan	rata_durasi	rata_tarif_per_menit
	0	0	82289	13.57
	1	1	57766	12.63
	2	2	40445	12.07
	3	3	26146	12.15
	4	4	16618	13.52
	5	5	16940	14.70
	6	6	42300	14.49
	7	7	84569	14.42
	8	8	114064	14.33
	9	9	127988	14.23
	10	10	140230	14.25
	11	11	150539	14.28
	12	12	165714	14.40
	13	13	174183	15.06
	14	14	186741	16.39
	15	15	191392	16.42
	16	16	190689	16.52
	17	17	204399	15.98
	18	18	210858	14.24
	19	19	188347	13.64
	20	20	161789	13.24
	21	21	157770	13.30
	22	22	143800	13.75
	23	23	111335	13.97

3. Pertanyaan 2: Perbedaan Hari Kerja dan Akhir Pekan

Untuk membandingkan hari kerja dan akhir pekan secara adil, saya menggunakan rata-rata perjalanan per hari karena jumlah hari yang teramati

pada kedua kelompok berbeda. Hari kerja memiliki 24 hari teramati, sedangkan akhir pekan memiliki 10 hari teramati.

```
... Ringkasan pola hari kerja dan akhir pekan:
```

	jenis_hari	hari_teramati	total_perjalanan	jam_puncak	jumlah_puncak	rata_perjalanan_per_hari_puncak	jam_terendah	jumlah_terendah	rata_durasi_saas_puncak	tarif_per_minut_saas_puncak
0	Hari kerja	24	2132583	18	159457	6644.04	4	6192	14.03	2.22
1	Akhir pekan	10	854328	16	54057	5405.70	5	4478	16.10	1.92

Pola berbeda: puncak hari kerja terjadi pada jam 18.00 (6,644.04 perjalanan/hari), sedangkan akhir pekan pada jam 16.00 (5,405.70 perjalanan/hari).

Berdasarkan hasil tersebut, pola permintaan berbeda antara hari kerja dan akhir pekan. Pada hari kerja, permintaan tertinggi terjadi pada jam 18.00 dengan rata-rata 6.644,04 perjalanan per hari. Sementara itu, pada akhir pekan, permintaan tertinggi bergeser ke jam 16.00 dengan rata-rata 5.405,70 perjalanan per hari. Pergeseran ini menunjukkan bahwa aktivitas perjalanan pada akhir pekan cenderung terjadi lebih awal dibandingkan hari kerja.

4. Pertanyaan 3: Perbandingan Jam Sibuk dan Jam Lain

Berdasarkan jumlah perjalanan tertinggi, jam sibuk ditentukan sebagai jam 15.00, 16.00, 17.00, dan 18.00. Hasil perbandingan antara jam sibuk dan jam lainnya adalah sebagai berikut.

```
... Jam sibuk: [15, 16, 17, 18].
Durasi rata-rata: jam sibuk 15.76 menit vs jam lain 14.15 menit; lebih pendek? Tidak.
Tarif kotor per menit: jam sibuk 2.05 vs jam lain 2.14; lebih tinggi? Tidak.
```

	kelompok_jam	jumlah_perjalanan	rata_durasi	rata_tarif_per_minut
0	Jam lain	2189573	14.15	2.14
1	Jam sibuk	797338	15.76	2.05

Berdasarkan hasil tersebut, perjalanan pada jam sibuk tidak lebih pendek daripada perjalanan pada jam lain. Rata-rata durasi perjalanan pada jam sibuk adalah 15,76 menit, sedangkan pada jam lain hanya 14,15 menit. Selain itu, tarif kotor per menit pada jam sibuk sebesar 2,05 lebih rendah dibandingkan jam lain yang mencapai 2,14. Dengan demikian, jam sibuk tidak terbukti lebih pendek dan tidak terbukti lebih menguntungkan per menit berdasarkan data ini.

Namun perlu diperhatikan bahwa `total_amount/durasi_minut` hanya menunjukkan tarif atau pendapatan kotor per menit, bukan laba bersih. Biaya

bahan bakar, gaji pengemudi, perawatan kendaraan, dan waktu tanpa penumpang belum diperhitungkan.

5. Rekomendasi Operasional

Berdasarkan hasil studi kasus, diberikan tiga rekomendasi operasional.

1. Operator sebaiknya memprioritaskan penambahan armada mendekati jam 18.00 karena permintaan tertinggi mencapai 210.858 perjalanan, dengan keterbatasan bahwa data hanya berasal dari satu file bulan Januari sehingga pola musiman belum terwakili.
2. Operator sebaiknya mengatur shift akhir pekan secara berbeda dengan memberikan perhatian lebih besar sekitar jam 16.00 karena rata-rata permintaannya mencapai 5.406 perjalanan per hari, dengan keterbatasan bahwa jumlah hari yang teramati terbatas sehingga pola ini perlu divalidasi menggunakan beberapa bulan tambahan.
3. Operator tidak sebaiknya menjadikan jam sibuk sebagai satu-satunya indikator keuntungan karena tarif kotor per menit pada jam sibuk hanya 2,05 dibandingkan 2,14 pada jam lain, dengan keterbatasan bahwa total_amount belum dikurangi biaya operasional dan waktu kendaraan tanpa penumpang.

G. Latihan

1. Latihan 1: Hitung 5 nilai trip_distance terbesar pada data bersih dan tampilkan baris lengkapnya. Apakah baris-baris itu masuk akal? Beri komentar satu kalimat.

```
[ ] top5_jarak = bersih.nlargest(5, "trip_distance").copy()
    print("Lima trip_distance terbesar pada data bersih:")
    top5_jarak
```

... Lima trip_distance terbesar pada data bersih:

	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	payment_type	fare_amount	tip_amount	total_amount	durasi menit	jam
1748933	2023-01-19 16:29:36	2023-01-19 18:46:01	1.0	96.70	1	350.000000	50.0	440.750000	136.416667	16
526988	2023-01-07 04:29:56	2023-01-07 06:06:16	2.0	93.05	2	542.000000	0.0	564.049988	96.333333	4
2402193	2023-01-26 04:55:45	2023-01-26 07:18:50	1.0	92.60	1	262.500000	20.0	312.799988	143.083333	4
2902389	2023-01-31 07:04:24	2023-01-31 09:32:54	1.0	92.00	1	334.100006	10.0	368.149994	148.500000	7
927215	2023-01-11 14:23:55	2023-01-11 16:17:58	1.0	90.63	2	250.000000	0.0	272.299988	114.050000	14

Kelima baris masih memenuhi aturan pembersihan (`0 < trip_distance < 100`), durasi 1-180 menit, dan `total_amount > 0`), tetapi jaraknya jauh di atas pola mayoritas sehingga layak dianggap sebagai perjalanan luar biasa dan diperiksa lebih lanjut, bukan langsung dihapus.

2. Latihan 2: Ulangi J-7 dengan `chunksize=100_000` lalu `chunksize=1_000_000`. Catat waktu dan `delta_rss_mb` keduanya, dan jelaskan arah trade-off-nya.

```
[ ]
hasil_chunking = []
for ukuran in [100_000, 1_000_000]:
    hasil = ukur(
        f"agregasi chunking {ukuran:}",
        lambda ukuran=ukuran: agregasi_bertahap(CSV, ukuran))
    hasil_chunking.append({
        "chunksize": ukuran,
        "baris": hasil[0],
        "total_amount": hasil[1],
        **catatan[-1]
    })

perbandingan_chunking = pd.DataFrame(hasil_chunking)
perbandingan_chunking
```

▼

[agregasi chunking 100,000] 4.52 s | RSS +5.0 MB
[agregasi chunking 1,000,000] 4.62 s | RSS +0.7 MB

	chunksize	baris	total_amount	langkah	detik	delta_rss_mb
0	100000	2986911	81621456.74	agregasi chunking 100,000	4.52	5.0
1	1000000	2986911	81621456.74	agregasi chunking 1,000,000	4.62	0.7

`chunksize=100_000` memproses lebih banyak potongan sehingga penggunaan memori per potongan biasanya lebih kecil, tetapi overhead pembacaan dan penggabungan lebih besar. `chunksize=1_000_000` biasanya lebih cepat karena jumlah iterasi lebih sedikit, dengan konsekuensi delta RSS yang cenderung lebih besar.

3. Latihan 3: Buat kolom `persen_tip = tip_amount / fare_amount * 100`, tangani kasus `fare_amount == 0` dengan benar, lalu hitung rata-rata persen tip per `payment_type`. Apa dugaan Anda tentang perbedaan antar metode pembayaran?

```
[ ] ▶ import numpy as np

# fare_amount = 0 tidak boleh menghasilkan pembagian tak terdefinisi
denominator = bersih["fare_amount"].mask(bersih["fare_amount"] == 0)
bersih["persen_tip"] = bersih["tip_amount"].div(denominator).mul(100)

tip_per_payment = (bersih
    .groupby("payment_type", observed=False)
    .agg(jumlah_baris=("persen_tip", "size"),
        jumlah_fare_nol=("fare_amount", lambda s: (s == 0).sum()),
        rata_persen_tip=("persen_tip", "mean"))
    .round(2)
    .reset_index())

tip_per_payment
```

...	payment_type	jumlah_baris	jumlah_fare_nol	rata_persen_tip
0	0	64106	0	20.219999
1	1	2385064	11	25.910000
2	2	512987	102	0.000000
3	3	9184	39	0.020000
4	4	15570	28	0.150000

Payment type 1 (kartu kredit) cenderung memiliki persen tip lebih tinggi karena tip dapat ditambahkan langsung pada transaksi kartu, sedangkan payment type 2 (tunai) biasanya tercatat tanpa tip; perbedaan ini menggambarkan cara pencatatan pembayaran, bukan bukti bahwa pengguna tunai sama sekali tidak memberi tip.

4. Latihan 4: Simpan bersih sebagai Parquet dengan dua kompresi berbeda (compression="snappy" dan "gzip"). Bandingkan ukuran file dan waktu tulisnya.

```
[ ]
hasil_kompresi = []
for kompresi in ["snappy", "gzip"]:
    path_kompresi = os.path.join(
        DIR_SIMPAN, f"bersih_{kompresi}.parquet")
    mulai = time.perf_counter()
    bersih.to_parquet(path_kompresi, index=False, compression=kompresi)
    detik = time.perf_counter() - mulai
    ukuran_mb = os.path.getsize(path_kompresi) / 1024**2
    hasil_kompresi.append({
        "compression": kompresi,
        "waktu_tulis_detik": round(detik, 2),
        "ukuran_mb": round(ukuran_mb, 2),
        "path": path_kompresi
    })

perbandingan_kompresi = pd.DataFrame(hasil_kompresi)
perbandingan_kompresi
```

	compression	waktu_tulis_detik	ukuran_mb	path
0	snappy	4.52	58.53	/content/drive/MyDrive/BigData/Praktikum1/bers...
1	gzip	45.13	46.30	/content/drive/MyDrive/BigData/Praktikum1/bers...

File gzip biasanya menghasilkan file lebih kecil karena kompresinya lebih agresif, tapi waktu tulisnya lebih lama daripada snappy. Snappy lebih cocok saat kecepatan baca/tulis dan pemrosesan ulang lebih penting daripada penghematan ruang maksimum.

5. Latihan 5: Tulis ulang agregasi per jam J-9 memakai Spark SQL, lalu buktikan bahwa hasil keduanya konsisten dengan membandingkan kolom jumlah secara terprogram (mis. selisih maksimum = 0). Jika ada selisih, jelaskan sumbernya.

```
[ ] spark_latihan = (SparkSession.builder
    .appName("BD-P01-Latihan-5")
    .master("local[*]")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate())
spark_latihan.sparkContext.setLogLevel("WARN")

sdf_raw_latihan = spark_latihan.read.parquet(PATH)
sdf_raw_latihan.createOrReplaceTempView("trips_raw_latihan")
hasil_sql = spark_latihan.sql("""
    WITH trips_durasi AS (
        SELECT *,
            (unix_timestamp(tpep_dropoff_datetime)
             - unix_timestamp(tpep_pickup_datetime)) / 60.0
            AS durasi menit
        FROM trips_raw_latihan
    ),
    trips_bersih_sql AS (
        SELECT *
        FROM trips_durasi
        WHERE trip_distance > 0
            AND trip_distance < 100
            AND durasi menit BETWEEN 1 AND 180
            AND total_amount > 0
        )
    SELECT hour(tpep_pickup_datetime) AS jam,
           COUNT(*) AS jumlah_perjalanan
    FROM trips_bersih_sql
    GROUP BY hour(tpep_pickup_datetime)
    ORDER BY jam
    """)
hasil_sql_pd = hasil_sql.toPandas()

perbandingan_sql = (agregasi[["jam", "jumlah_perjalanan"]]
    .merge(hasil_sql_pd, on="jam", suffixes=("_pandas", "_spark_sql")))
perbandingan_sql["selisih"] = (
    perbandingan_sql["jumlah_perjalanan_pandas"]
    - perbandingan_sql["jumlah_perjalanan_spark_sql"])
maks_selisih = int(perbandingan_sql["selisih"].abs().max())
print("Maksimum selisih jumlah:", maks_selisih)
display(perbandingan_sql)
assert maks_selisih == 0, "Hasil pandas dan Spark SQL belum konsisten"
spark_latihan.stop()
```

▼ ... Maksimum selisih jumlah: 0

	jam	jumlah_perjalanan_pandas	jumlah_perjalanan_spark_sql	selisih	
0	0	82289	82289	0	
1	1	57766	57766	0	
2	2	40445	40445	0	
3	3	26146	26146	0	
4	4	16618	16618	0	
5	5	16940	16940	0	
6	6	42300	42300	0	
7	7	84569	84569	0	
8	8	114064	114064	0	
9	9	127988	127988	0	
10	10	140230	140230	0	
11	11	150539	150539	0	
12	12	165714	165714	0	
13	13	174183	174183	0	
14	14	186741	186741	0	
15	15	191392	191392	0	
16	16	190689	190689	0	
17	17	204399	204399	0	
18	18	210858	210858	0	
19	19	188347	188347	0	
20	20	161789	161789	0	
21	21	157770	157770	0	
22	22	143800	143800	0	
23	23	111335	111335	0	

Jika 'maks_selisih = 0', agregasi pandas dan Spark SQL konsisten karena memakai kolom waktu serta threshold pembersihan yang sama. Jika muncul selisih, sumber yang perlu diperiksa adalah timezone, pembulatan durasi, atau perbedaan filter; kode di atas menggunakan hitungan baris tanpa pembulatan sehingga seharusnya menghasilkan selisih nol.

H. Pertanyaan Evaluasi

1. Jelaskan mengapa “ukuran data” saja tidak cukup untuk menyebut suatu masalah sebagai masalah Big Data. Berikan satu contoh data kecil yang tetap sulit diolah.

2. Dari 5V, dimensi mana yang paling menonjol pada dataset praktikum ini? Dukung jawaban Anda dengan satu angka hasil pengukuran.
3. Sebutkan dua alasan teknis mengapa Parquet lebih kecil dan lebih cepat dibaca daripada CSV untuk beban kerja analitik.
4. Anda punya CSV 40 GB dan mesin dengan RAM 1G GB. Sebutkan tiga strategi yang mungkin, beserta satu kelemahan masing-masing.
5. Apa yang dimaksud lazy evaluation pada Spark, dan bagaimana hal itu memengaruhi cara Anda mengukur waktu eksekusi secara benar?
6. Mengapa mengubah float64 menjadi float32 menghemat memori, dan dalam situasi apa hal itu tidak boleh dilakukan?
7. Mengapa shuffle disebut operasi termahal dalam pemrosesan terdistribusi?
8. Mengapa file di /content pada Colab tidak boleh dijadikan tempat penyimpanan akhir, dan apa konsekuensinya bagi desain notebook Anda?

Jawab:

1. Ukuran data saja tidak cukup untuk menentukan apakah suatu masalah termasuk Big Data. Data yang berukuran kecil tetap dapat sulit diolah apabila memiliki format yang tidak terstruktur, membutuhkan pemrosesan dengan latensi rendah, memiliki kualitas yang buruk, atau mengandung informasi sensitif. Contohnya adalah data rekam medis berukuran 10 MB yang membutuhkan pemrosesan teks, perlindungan privasi, dan interpretasi yang kompleks. Sebaliknya, data berukuran lebih besar dapat tetap mudah diproses apabila strukturnya sederhana dan masih dapat dimuat ke memori.
2. Dimensi yang paling menonjol adalah Volume. Dataset Juli memiliki 2.907.108 baris dan pembacaan seluruh kolom membutuhkan memori DataFrame sebesar 502,5 MB. Besarnya jumlah baris tersebut membuat saya perlu menggunakan projection, optimasi tipe data, chunking, dan Spark local mode agar proses pengolahan lebih efisien.

3. Parquet lebih kecil karena menggunakan penyimpanan columnar, encoding, dan kompresi. Parquet juga lebih cepat untuk analisis karena hanya kolom yang diperlukan yang dibaca. Pada praktikum ini, Parquet Juli berukuran 46,1 MB, sedangkan CSV bersih berukuran 213,6 MB atau sekitar 4,63 kali lebih besar. CSV juga memerlukan proses parsing karena angka, tanggal, dan nilai lainnya disimpan dalam bentuk teks.
4. Strategi pertama yang dapat saya gunakan adalah chunking, yaitu membaca CSV secara bertahap agar seluruh file tidak dimuat ke memori. Kelemahannya adalah kode menjadi lebih kompleks dan tidak semua agregasi dapat digabungkan dengan mudah. Strategi kedua adalah menggunakan DuckDB, Polars, atau Spark untuk melakukan pemrosesan secara out-of-core. Kelemahannya adalah terdapat overhead konfigurasi dan pemrosesan. Strategi ketiga adalah mengonversi CSV menjadi Parquet terpartisi agar proses berikutnya lebih efisien. Kelemahannya adalah proses konversi awal membutuhkan waktu dan ruang penyimpanan sementara.
5. Lazy evaluation berarti Spark tidak langsung menjalankan transformasi seperti filter, withColumn, dan groupBy. Spark hanya membangun rencana eksekusi terlebih dahulu. Eksekusi baru dilakukan ketika action seperti count(), show(), collect(), atau write() dipanggil. Oleh karena itu, saya harus mengukur waktu pada saat action dijalankan, bukan hanya mengukur waktu ketika transformasi ditulis. Pada notebook, waktu Spark count tercatat 2,86 detik dan Spark agregasi tercatat 5,96 detik.
6. float64 menggunakan 8 byte untuk setiap nilai, sedangkan float32 menggunakan 4 byte. Oleh karena itu, perubahan dari float64 menjadi float32 dapat menghemat sekitar setengah memori kolom numerik. Pada dataset Juli, optimasi tipe data menurunkan memori dari 177,4 MB menjadi 113,7 MB atau menghemat 35,9%. Namun, perubahan ini tidak sebaiknya

dilakukan pada perhitungan tagihan, saldo, audit keuangan, atau perhitungan lain yang membutuhkan presisi tinggi karena float32 memiliki presisi yang lebih rendah.

7. Shuffle mahal karena data harus dipindahkan dari satu partisi ke partisi lain berdasarkan kunci tertentu, misalnya pada groupBy atau join. Proses tersebut membutuhkan redistribusi data, serialisasi, sinkronisasi, dan terkadang penulisan sementara ke disk. Selain itu, data yang tidak seimbang dapat menyebabkan salah satu partisi bekerja lebih lama daripada partisi lainnya. Pada hasil explain() Spark, operasi shuffle terlihat dari bagian Exchange
8. File di /content bersifat sementara dan dapat hilang ketika runtime Colab dihentikan, di-reset, atau mengalami idle terlalu lama. Oleh karena itu, file tersebut tidak boleh digunakan sebagai penyimpanan akhir. Konsekuensinya, notebook harus menghubungkan Google Drive, membaca dataset dari lokasi yang jelas, menyimpan hasil CSV, Parquet, dan PNG ke Drive, serta memiliki kode yang dapat menyiapkan ulang lingkungan ketika dijalankan dari runtime baru.

I. Kesimpulan

Praktikum berhasil menunjukkan bahwa pemilihan kolom, optimasi tipe data, penggunaan Parquet, dan chunking dapat menghemat memori dalam pengolahan jutaan baris data. Dataset Januari memiliki 3.066.766 baris mentah dan setelah dibersihkan tersisa 2.986.911 baris. Agregasi pandas dan Spark SQL juga menghasilkan jumlah yang konsisten.

Berdasarkan studi kasus, permintaan tertinggi terjadi pada jam 18.00, sedangkan permintaan terendah terjadi pada jam 04.00. Puncak akhir pekan bergeser ke jam 16.00, tetapi jam sibuk tidak terbukti lebih pendek maupun lebih menguntungkan per menit. Dimensi 5V yang paling menonjol adalah Volume, sedangkan Spark belum lebih cepat daripada pandas pada ukuran data ini.

Github : [BigData26_A_2411533006_AlfarisAuliaRahman](#)

Drive : [BigData](#)